UNIVERSIDAD MARIANO GÁLVEZ

SEDE: CHIMALTENANGO, COLEGIO MESOAMERICANO

FACTULTAD DE INGENIERÍA EN SISTEMAS

CURSO: 027 – BASES DE DATOS I

INGENIERO HÉCTOR JAHAZIEL LUNA IRRIATE

MANUAL TÉCNICO – SISTEMA DE APUESTAS DEPORTIVAS Y PRONÓSTICOS

FECHA: 31 DE OCTUBRE DE 2026

AJUCHÁN POPOL, ALLAN ANTHONY – 1990-24-13391

GIRÓN MORALES, OSCAR EDUARDO – 1990-24-12952

MOX SUJUY, DIEGO JULIO ALBERTO – 1990-24-14979

PICHIYÁ APÉN, JOSÉ NEHEMÍAS – 1990-24-25116

ÍNDICE

INTRODUCCIÓN Y ALCANCE DEL DOCUMENTO 1
ARQUITECTURA GENERAL 2
MODELO DE DATOS 4
SEGURIDAD IMPLEMENTADA 15
LÓGICA DE NEGOCIO CRÍTICA 19
ESTRUCTURA DE RUTAS (URLs) PRINCIPALES 21
FORMULARIOS Y REGLAS DE VALIDACIÓN 23
CONFIGURACIÓN DEL PROYECTO (settings.py) 24
INSTALACIÓN Y DESPLIEGUE 25
PRUEBAS REALIZADAS DURANTE EL DESARROLLO 28
MANTENIMIENTO Y EXTENSIÓN FUTURA 29

INTRODUCCIÓN Y ALCANCE DEL DOCUMENTO

Este documento describe, con detalle técnico, la arquitectura, el modelo de datos, la lógica de negocio, los mecanismos de seguridad, la estructura de código y el procedimiento de instalación y despliegue de la Plataforma de Apuestas Deportivas y Pronósticos. Está dirigido a desarrolladores, administradores de bases de datos y personal técnico responsable de instalar, mantener, auditar o extender el sistema.

El sistema fue desarrollado en Python utilizando el framework Django, con persistencia de datos en SQL Server. La arquitectura sigue el patrón habitual de Django (modelo-vista-plantilla), organizada en aplicaciones (apps) independientes según el dominio funcional que atienden.

Este manual técnico cubre los siguientes aspectos del sistema:

- Arquitectura general y tecnologías utilizadas.

- Organización del código en aplicaciones Django y responsabilidad de cada una.

- Modelo de datos completo: las 38 tablas del sistema, sus campos, tipos de dato y relaciones.

- Mecanismos de seguridad implementados a nivel de aplicación, sesión, base de datos y perímetro (middleware).

- Lógica de negocio crítica: creación de boletos, liquidación de eventos y pago automático, gestión de retiros.

- Rutas (URLs) y vistas principales del sistema.

- Formularios y reglas de validación aplicadas.

- Configuración del proyecto (settings.py) y variables de entorno.

- Procedimiento completo de instalación, configuración de base de datos y despliegue.

- Comandos de gestión incluidos y su propósito.

- Pruebas realizadas durante el desarrollo y recomendaciones de mantenimiento.

ARQUITECTURA GENERAL

2.1 PILA TECNOLÓGICA

- Lenguaje: Python 3.12 – 3.14.

- Framework web: Django 4.2 (LTS).

- Motor de base de datos: SQL Server 2022 o superior, codificación utf8mb4.

- Conector de base de datos: mssql-python (driver nativo de Python para SQL Server).

- Gestión de variables de entorno: python-decouple.

- Interfaz de usuario: plantillas de Django (server-side rendering), HTML5 y CSS propio sin frameworks de terceros, con diseño responsivo basado en CSS Grid y Flexbox, más una pequeña cantidad de JavaScript nativo (sin librerías externas) para la interacción dinámica del armado de boletos y la carga en cascada de municipios.

- Autenticación: mecanismo propio basado en sesiones de Django, sin utilizar el modelo de usuario estándar de Django (auth.User), ya que el sistema define su propio esquema de login, usuario, administrador y super administrador.

2.2 PATRÓN DE DISEÑO

El sistema sigue el patrón modelo-vista-plantilla (MVT) propio de Django. Cada aplicación (app) define sus propios modelos (models.py), formularios (forms.py), vistas (views.py) y rutas (urls.py); las plantillas HTML se organizan en una carpeta global de templates, separadas por subcarpeta según la aplicación a la que pertenecen.

2.3 APLICACIONES (Apps) DEL PROYECTO

El proyecto Django principal se denomina apuestas_deportivas y contiene las siguientes seis aplicaciones internas:

- Catálogos: Contiene los catálogos generales y de referencia del sistema: país, departamento, municipio, estado (catálogo genérico de estados usado por distintas entidades), deporte, liga, equipo/atleta, tipo de mercado y método de pago. Estas tablas no dependen de ninguna otra aplicación y son la base sobre la que se construyen las demás.

- Cuentas: Contiene toda la lógica de autenticación y gestión de cuentas: login, usuario final, administrador, super administrador, preguntas de seguridad, recuperación de contraseña, bitácoras de sesión y de acciones administrativas, bloqueos de cuenta y registro de intentos de inicio de sesión. Incluye además las utilidades de autenticación propias (auth_utils.py), el

middleware de control de acceso por rol, y el context processor que expone los datos de la sesión activa a todas las plantillas.

- Billetera: Contiene la billetera virtual de cada usuario, las transacciones financieras, las cuentas bancarias registradas por los usuarios, las solicitudes de retiro y el historial detallado de movimientos de saldo.

- Apuestas: Contiene el núcleo funcional del negocio: eventos deportivos, resultados oficiales, mercados de apuesta, opciones de apuesta, promociones de cuota, boletos y su detalle, así como los historiales de cambios de cuota, de estado de boleto, de cambios de evento y el resumen mensual de actividad por jugador. Incluye también la lógica de creación de boletos y de liquidación automática de eventos.

- Auditoria: Contiene la bitácora de seguridad del sistema y el middleware perimetral encargado de detectar y bloquear patrones de inyección SQL/XSS, así como de aplicar el límite de solicitudes por dirección IP (mitigación de ataques de denegación de servicio a nivel de aplicación).

- Panel Público: Contiene la vista pública del sistema, accesible sin autenticación, que muestra la cartelera de eventos del día en curso y los resultados del día anterior.

MODELO DE DATOS

El modelo de datos conserva los nombres y relaciones del diseño original proporcionado por el cliente, utilizando el prefijo tbl_ y nomenclatura en español (snake_case) para todas las tablas y columnas. En total, el modelo está compuesto por 38 tablas, descritas a continuación agrupadas por dominio funcional. PK indica llave primaria, FK indica llave foránea y UK indica restricción de unicidad.

3.1 CATÁLOGOS GENERALES

tbl_pais — catálogo de países.

id_pais (INT, PK) — identificador autoincremental.

nombre_pais (VARCHAR(100)) — nombre del país.

codigo_iso (VARCHAR(5)) — código ISO del país.

tbl_departamento — departamentos de un país.

id_departamento (INT, PK) — identificador autoincremental.

id_pais (INT, FK → tbl_pais) — país al que pertenece.

nombre_departamento (VARCHAR(100)) — nombre del departamento.

comentario_departamento (VARCHAR(255), opcional) — nota administrativa.

tbl_municipio — municipios de un departamento.

id_municipio (INT, PK) — identificador autoincremental.

id_departamento (INT, FK → tbl_departamento) — departamento al que pertenece.

nombre_municipio (VARCHAR(100)) — nombre del municipio.

comentario_municipio (VARCHAR(255), opcional) — nota administrativa.

tbl_estado — catálogo genérico de estados, reutilizado por deporte, usuario, administrador, billetera, etc.

id_estado (INT, PK) — identificador autoincremental.

nombre_estado (VARCHAR(50)) — por ejemplo ACTIVO, INACTIVO, BLOQUEADO, ACTIVA, SUSPENDIDA.

comentario_estado (VARCHAR(255), opcional) — nota administrativa.

tbl_deporte — disciplinas deportivas soportadas.

id_deporte (INT, PK) — identificador autoincremental.

nombre_deporte (VARCHAR(100)) — por ejemplo Fútbol, Baloncesto.

id_estado (INT, FK → tbl_estado) — estado del deporte dentro del catálogo.

tbl_liga — ligas o torneos de un deporte en un país.

id_liga (INT, PK) — identificador autoincremental.

id_deporte (INT, FK → tbl_deporte) — deporte al que pertenece la liga.

id_pais (INT, FK → tbl_pais) — país de origen de la liga.

nombre_liga (VARCHAR(150)) — nombre de la liga.

tbl_equipo_atleta — equipos o atletas individuales.

id_equipo (INT, PK) — identificador autoincremental.

nombre_equipo (VARCHAR(150)) — nombre del equipo o atleta.

id_deporte (INT, FK → tbl_deporte) — deporte que practica.

tbl_tipo_mercado — tipos de mercado de apuesta disponibles (plantillas reutilizables entre eventos).

id_tipo_mercado (INT, PK) — identificador autoincremental.

nombre_mercado (VARCHAR(100)) — por ejemplo Ganador del partido, Total de goles/puntos.

tbl_metodo_pago — métodos de pago disponibles para depósitos.

id_metodo_pago (INT, PK) — identificador autoincremental.

nombre_metodo (VARCHAR(100)) — por ejemplo tarjeta, transferencia, billetera electrónica.

comision_porcentaje (DECIMAL(5,2)) — comisión aplicada por el método, informativa.

3.2 CUENTAS, LOGIN Y SEGURIDAD DE ACCESO

tbl_login — credenciales únicas del sistema; un login puede corresponder a un usuario final, un administrador o un super administrador, según tipo_usuario.

id_login (INT, PK) — identificador autoincremental.

id_sistema_us (VARCHAR(50), UK) — identificador único de sistema (correo o usuario).

nombre_usuario (VARCHAR(100), UK) — nombre de usuario para inicio de sesión.

contrasena_usuario (VARCHAR(255)) — hash de la contraseña; nunca se almacena en texto plano.

tipo_usuario (ENUM) — USUARIO, ADMIN o SUPER_ADMIN.

fecha_registro_login (DATETIME) — fecha de creación de la cuenta.

intentos_fallidos (SMALLINT) — contador de intentos fallidos consecutivos.

bloqueado_hasta (DATETIME, opcional) — fecha/hora hasta la cual la cuenta permanece bloqueada.

ultimo_cambio_password (DATETIME, opcional) — fecha del último cambio de contraseña.

tbl_login_log — bitácora básica de accesos por login.

id_login_log (INT, PK) —

id_login (INT, FK → tbl_login) —

fecha_registro_log_us (DATETIME) —

ip_acceso (VARCHAR(45)) — dirección IP de origen.

tbl_usuario — datos personales del usuario final (apostador).

id_usuario (INT, PK) —

id_login (INT, FK → tbl_login, UK) — relación uno a uno con el login.

primer_nombre_us / segundo_nombre_us (VARCHAR(80)) — segundo nombre opcional.

primer_apellido_us / segundo_apellido_us / apellido_casada_us (VARCHAR(80)) — los dos últimos opcionales.

fecha_nacimiento_us (DATE) — validada como mayor de edad en el formulario.

id_departamento_us (INT, FK → tbl_departamento) —

id_municipio_us (INT, FK → tbl_municipio) —

direccion_us (VARCHAR(255)) —

correo_us (VARCHAR(254), UK) — correo electrónico único.

telefono_us (VARCHAR(20)) —

id_estado (INT, FK → tbl_estado) —

tbl_administrador — datos del personal administrativo (no super administrador).

id_admin (INT, PK) —

id_login (INT, FK → tbl_login, UK) —

codigo_empleado (VARCHAR(30), UK) —

primer_nombre_admin / primer_apellido_admin (VARCHAR(80)) —

departamento_empresa (VARCHAR(100)) —

correo_corporativo (VARCHAR(254)) —

rol_administrativo (ENUM) — OPERADOR_EVENTOS, CAJERO o AUDITOR.

id_estado (INT, FK → tbl_estado) —

tbl_super_administrador — datos del super administrador del sistema.

id_super_admin (INT, PK) —

id_login (INT, FK → tbl_login, UK) —

nombre_completo (VARCHAR(150)) —

correo_maestro (VARCHAR(254)) —

llave_mfa_secret (VARCHAR(64), opcional) — reservado para una futura autenticación de doble factor.

fecha_asignacion (DATETIME) —

tbl_pregunta_seguridad (NUEVA) — catálogo de preguntas de seguridad disponibles para la recuperación de contraseña.

id_pregunta (INT, PK) —

texto_pregunta (VARCHAR(255)) —

tbl_usuario_pregunta_seguridad (NUEVA) — respuesta cifrada que cada login configuró para una pregunta de seguridad.

id_usuario_pregunta (INT, PK) —

id_login (INT, FK → tbl_login) —

id_pregunta (INT, FK → tbl_pregunta_seguridad) —

respuesta_hash (VARCHAR(255)) — hash de la respuesta; combinación única con id_login.

tbl_recuperacion_password (NUEVA) — token temporal de un solo uso para restablecer contraseña.

id_recuperacion (INT, PK) —

id_login (INT, FK → tbl_login) —

token (CHAR(36), UK) — UUID generado aleatoriamente.

fecha_solicitud / fecha_expiracion (DATETIME) — vigencia de 30 minutos desde la solicitud.

usado (TINYINT(1)) — indica si el token ya fue consumido.

ip_solicitud (VARCHAR(45), opcional) —

tbl_admin_log — bitácora de acciones administrativas relevantes.

id_admin_log (BIGINT, PK) —

id_login (INT, FK → tbl_login) —

accion_realizada (VARCHAR(255)) —

tabla_afectada (VARCHAR(100)) —

fecha_registro (DATETIME) —

tbl_historial_sesiones — registro de cada intento de inicio de sesión, exitoso o fallido.

id_historial_sesion (BIGINT, PK) —

id_login (INT, FK → tbl_login) —

ip_origen (VARCHAR(45)) —

dispositivo_navegador (VARCHAR(255)) —

resultado (ENUM) — EXITOSO o FALLIDO.

fecha_conexion (DATETIME) —

tbl_historial_bloqueos_cuenta — historial de bloqueos aplicados sobre la cuenta de un usuario.

id_historial_bloqueo (INT, PK) —

id_usuario (INT, FK → tbl_usuario) —

motivo_bloqueo (VARCHAR(255)) —

solicitado_por (ENUM) — SISTEMA, ADMIN o USUARIO.

fecha_inicio_bloqueo / fecha_fin_bloqueo (DATETIME) —

tbl_intento_login (NUEVA) — registro de todo intento de acceso por dirección IP, usado para mitigar fuerza bruta.

id_intento (BIGINT, PK) —

id_sistema_us (VARCHAR(50), opcional) —

ip_origen (VARCHAR(45)) —

exitoso (TINYINT(1)) —

fecha_intento (DATETIME) — indexado junto a ip_origen y a id_sistema_us.

3.3 BILLETERA VIRTUAL Y MOVIMIENTOS FINANCIEROS

tbl_billetera — billetera virtual de cada usuario.

id_billetera (INT, PK) —

id_usuario (INT, FK → tbl_usuario, UK) — relación uno a uno.

saldo_disponible (DECIMAL(14,2)) — restricción CHECK ≥ 0.

saldo_retenido (DECIMAL(14,2)) — restricción CHECK ≥ 0; dinero en apuestas pendientes.

id_estado (INT, FK → tbl_estado) —

tbl_transaccion — registro de cada transacción financiera individual.

id_transaccion (BIGINT, PK) —

id_billetera (INT, FK → tbl_billetera) —

id_metodo_pago (INT, FK → tbl_metodo_pago, opcional) —

tipo_transaccion (ENUM) — DEPOSITO, RETIRO, APUESTA, PREMIO o AJUSTE.

monto (DECIMAL(14,2)) — restricción CHECK > 0.

fecha_hora (DATETIME) —

tbl_cuenta_bancaria_usuario — cuentas bancarias registradas por el usuario para recibir retiros.

id_cuenta_bancaria (INT, PK) —

id_usuario (INT, FK → tbl_usuario) —

nombre_banco (VARCHAR(100)) —

numero_cuenta (VARCHAR(40)) —

tipo_cuenta (ENUM) — MONETARIA o AHORRO.

tbl_solicitud_retiro — solicitudes de retiro realizadas por los usuarios.

id_solicitud (INT, PK) —

id_billetera (INT, FK → tbl_billetera) —

id_cuenta_bancaria (INT, FK → tbl_cuenta_bancaria_usuario) —

monto_solicitado (DECIMAL(14,2)) — restricción CHECK > 0.

estado_solicitud (ENUM) — PENDIENTE, APROBADA, RECHAZADA o PAGADA.

fecha_solicitud (DATETIME) —

tbl_historial_saldo_billetera — historial detallado de cada movimiento de saldo de una billetera.

id_historial_saldo (BIGINT, PK) —

id_billetera (INT, FK → tbl_billetera) —

saldo_anterior / saldo_nuevo (DECIMAL(14,2)) —

monto_movimiento (DECIMAL(14,2)) —

tipo_operacion (ENUM) — DEPOSITO, RETIRO, DEBITO_APUESTA, CREDITO_PREMIO, RETENCION, LIBERACION o AJUSTE_ADMIN.

fecha_movimiento (DATETIME) —

3.4 EVENTOS, MERCADOS, OPCIONES Y BOLETOS

bl_evento — eventos deportivos programados.

id_evento (INT, PK) —

id_liga (INT, FK → tbl_liga) —

id_equipo_local / id_equipo_visitante (INT, FK → tbl_equipo_atleta) —

fecha_inicio (DATETIME) — indexado.

estado_evento (ENUM) — PROGRAMADO, EN_JUEGO, FINALIZADO, SUSPENDIDO o CANCELADO.

tbl_resultado_evento — resultado oficial de un evento ya finalizado.

id_resultado (INT, PK) —

id_evento (INT, FK → tbl_evento, UK) — relación uno a uno.

puntuacion_local / puntuacion_visitante (INT sin signo) —

observaciones (VARCHAR(255), opcional) —

fecha_registro (DATETIME) —

tbl_mercado_evento — mercado de apuesta habilitado para un evento específico.

id_mercado_evento (INT, PK) —

id_evento (INT, FK → tbl_evento) —

id_tipo_mercado (INT, FK → tbl_tipo_mercado) — combinación única con id_evento.

estado_mercado (ENUM) — ABIERTO, CERRADO o LIQUIDADO.

tbl_opcion_apuesta — opción concreta de apuesta dentro de un mercado, con su cuota.

id_opcion (INT, PK) —

id_mercado_evento (INT, FK → tbl_mercado_evento) —

descripcion_opcion (VARCHAR(150)) —

cuota_actual (DECIMAL(6,2)) — restricción CHECK > 1.00.

es_ganadora (TINYINT(1)) — se define en el proceso de liquidación.

tbl_promocion_cuota — incremento temporal (boost) aplicable a una opción de apuesta.

id_promocion (INT, PK) —

id_opcion (INT, FK → tbl_opcion_apuesta) —

porcentaje_aumento (DECIMAL(5,2)) — restricción CHECK ≥ 0.

fecha_inicio / fecha_fin (DATETIME) — vigencia de la promoción.

tbl_boleto — boleto de apuesta creado por un usuario (simple o compuesto).

id_boleto (BIGINT, PK) —

id_billetera (INT, FK → tbl_billetera) —

monto_apostado (DECIMAL(14,2)) — restricción CHECK > 0.

cuota_total (DECIMAL(8,2)) — producto de las cuotas de todas las selecciones.

ganancia_potencial (DECIMAL(14,2)) — restricción CHECK ≥ 0; monto_apostado × cuota_total.

estado_boleto (ENUM) — PENDIENTE, GANADO, PERDIDO, ANULADO o PAGADO.

fecha_emision (DATETIME) —

tbl_detalle_boleto — cada selección individual que compone un boleto.

id_detalle (BIGINT, PK) —

id_boleto (BIGINT, FK → tbl_boleto) —

id_opcion (INT, FK → tbl_opcion_apuesta) — combinación única con id_boleto.

cuota_momento (DECIMAL(6,2)) — cuota vigente al momento de apostar (se congela aquí).

3.5 HISTORIALES Y BITÁCORAS DE AUDITORÍA

tbl_historial_cuotas — historial de cada cambio de cuota de una opción de apuesta.

id_historial_cuota (BIGINT, PK) —

id_opcion (INT, FK → tbl_opcion_apuesta) —

cuota_anterior / cuota_nueva (DECIMAL(6,2)) —

fecha_cambio (DATETIME) —

id_admin (INT, FK → tbl_administrador, opcional) — administrador responsable del cambio.

tbl_historial_estado_boleto — historial de cada cambio de estado de un boleto.

id_historial_boleto (BIGINT, PK) —

id_boleto (BIGINT, FK → tbl_boleto) —

estado_anterior / estado_nuevo (ENUM) —

motivo_cambio (VARCHAR(255), opcional) —

fecha_cambio (DATETIME) —

tbl_historial_cambios_evento — historial de cambios de estado de un evento (incluye datos en vivo).

id_historial_evento (BIGINT, PK) —

id_evento (INT, FK → tbl_evento) —

estado_anterior / estado_nuevo (ENUM) —

minuto_partido (SMALLINT sin signo, opcional) —

marcador_momento (VARCHAR(20), opcional) —

fecha_registro (DATETIME) —

tbl_historial_jugador_resumen — resumen mensual de actividad de apuestas por usuario.

id_resumen (INT, PK) —

id_usuario (INT, FK → tbl_usuario) — combinación única con mes_ano.

mes_ano (CHAR(7)) — formato AAAA-MM.

total_boletos_creados (INT sin signo) —

monto_total_apostado / monto_total_reclamado / rendimiento_neto (DECIMAL(14,2)) —

tbl_bitacora_seguridad (NUEVA) — bitácora de eventos de seguridad detectados por el middleware perimetral.

id_bitacora (BIGINT, PK) —

tipo_evento (ENUM) — SQLI, DDOS, VALOR_INVALIDO, RATE_LIMIT, ACCESO_NO_AUTORIZADO u OTRO.

ip_origen (VARCHAR(45)) — indexado junto a fecha_registro.

ruta_solicitada (VARCHAR(255)) —

detalle (TEXT, opcional) —

bloqueado (TINYINT(1)) —

fecha_registro (DATETIME) —

La totalidad de estas tablas, junto con sus llaves foráneas, restricciones CHECK e índices, se encuentra definida en el script sql/bd_apuestas_deportivas.sql el cual puede ejecutarse directamente sobre una instancia de SQL Server 2022 para crear la base de datos completa.

SEGURIDAD IMPLEMENTADA

4.1 GESTIÓN DE CONTRASEÑAS Y RESPUESTAS DE SEGURIDAD

Las contraseñas de todas las cuentas (tbl_login.contrasena_usuario) y las respuestas a las preguntas de seguridad (tbl_usuario_pregunta_seguridad.respuesta_hash) se almacenan exclusivamente como hash, utilizando las funciones make_password y check_password del framework Django (basadas en PBKDF2 con sal aleatoria por defecto). En ningún punto del sistema se almacena ni se transmite en texto plano una contraseña o respuesta de seguridad ya registrada.

4.2 MIDDLEWARE DE SEGURIDAD PERIMETRAL (auditoria.middleware.SeguridadPerimetralMiddleware)

Este middleware se ejecuta en cada solicitud HTTP recibida por el sistema, antes de que la petición llegue a la vista correspondiente, y realiza dos funciones:

- Detección de inyección SQL y de scripts (XSS): Revisa el contenido de todos los parámetros recibidos por GET y por POST contra una lista de expresiones regulares que identifican patrones típicos de ataques de inyección SQL (por ejemplo, comillas simples seguidas de operadores lógicos, palabras reservadas como UNION SELECT, INSERT INTO, DROP TABLE, DELETE FROM, comentarios SQL, llamadas a procedimientos almacenados) y de inyección de scripts (etiquetas <script>). Si se detecta alguna coincidencia, la solicitud se rechaza de inmediato con un código de respuesta HTTP 403 (prohibido), sin que la petición llegue a ejecutar ninguna consulta contra la base de datos, y el evento queda registrado en tbl_bitacora_seguridad con tipo_evento = SQLI.

- Límite de solicitudes por dirección IP (mitigación de DDOS): Utilizando el sistema de caché en memoria de Django, el middleware lleva un conteo de solicitudes recibidas por cada dirección IP durante una ventana de sesenta segundos. Si una misma IP supera el límite configurado en SEGURIDAD_MAX_PETICIONES_POR_MINUTO (60 solicitudes por minuto por defecto), las solicitudes adicionales son rechazadas con un código de respuesta HTTP 429 (demasiadas solicitudes), y el evento queda registrado en tbl_bitacora_seguridad con tipo_evento = DDOS.

Es importante señalar que, de manera independiente a este middleware, el uso del ORM (mapeo objeto-relacional) de Django en la totalidad de las consultas del sistema ya parametriza automáticamente todas las sentencias SQL generadas, por lo que el middleware descrito constituye una capa adicional de defensa en profundidad, y no el único mecanismo de protección contra inyección SQL.

4.3 MIDDLEWARE DE CONTROL DE ACCESO POR ROL (cuentas.middleware.ControlAccesoMiddleware)

Como segunda capa de protección, además de los decoradores aplicados individualmente a cada vista (ver sección 4.5), este middleware verifica, por prefijo de URL, que toda ruta bajo /panel-administrativo/ sea accedida únicamente por una sesión cuyo tipo_usuario sea ADMIN o SUPER_ADMIN, y que toda ruta bajo /panel-usuario/ sea accedida únicamente por una sesión cuyo tipo_usuario sea USUARIO. De esta manera, si en el futuro se agrega una nueva vista administrativa y por error no se protege individualmente con el decorador correspondiente, este middleware actúa como respaldo evitando que quede expuesta sin autenticación.

4.4 CONTROL DE INTENTOS FALLIDOS DE INICIO DE SESIÓN

Cada cuenta de login (tbl_login) mantiene un contador de intentos_fallidos. Cada vez que se recibe una contraseña incorrecta para una cuenta existente, el contador se incrementa en uno. Al alcanzar cinco intentos fallidos consecutivos, el sistema establece bloqueado_hasta en la fecha y hora actual más quince minutos (valor configurado en SEGURIDAD_MINUTOS_BLOQUEO_LOGIN), y reinicia el contador a cero. Mientras la fecha de bloqueo no haya transcurrido, cualquier intento de inicio de sesión sobre esa cuenta es rechazado, independientemente de que la contraseña ingresada sea correcta o no. Adicionalmente, cada intento (exitoso o fallido) queda registrado en tbl_intento_login (por dirección IP y usuario) y en tbl_historial_sesiones (por login, con el dispositivo/navegador utilizado).

4.5 AUTENTICACIÓN Y CONTROL DE ACCESO POR VISTA

El sistema no utiliza el modelo de usuario estándar de Django (auth.User) para la autenticación de negocio, dado que el esquema propio de tbl_login, tbl_usuario, tbl_administrador y tbl_super_administrador ya cumple ese propósito. En su lugar, el módulo cuentas/auth_utils.py implementa funciones propias: iniciar_sesion (que almacena en la sesión el id_login, el tipo_usuario y el nombre a mostrar), cerrar_sesion, obtener_login_actual, y el decorador requiere_sesion(tipos_permitidos), el cual se aplica sobre cada vista que requiere autenticación, verificando que el tipo_usuario almacenado en la sesión activa se encuentre dentro de la lista de tipos permitidos para esa vista en particular (por ejemplo, ['USUARIO'] para las vistas del panel de usuario, o ['ADMIN', 'SUPER_ADMIN'] para las vistas del panel administrativo).

Dentro del panel administrativo, además, cada vista verifica adicionalmente el rol_administrativo específico del administrador autenticado (operador de eventos, cajero o

auditor) antes de permitir la acción solicitada; por ejemplo, la vista de liquidación de eventos únicamente permite continuar si el rol es OPERADOR_EVENTOS o si la sesión corresponde a un SUPER_ADMIN.

4.6 VALIDACIÓN DE MONTOS Y CUOTAS

Tanto a nivel de formularios (Django Forms) como a nivel de base de datos (restricciones CHECK), el sistema impide el registro de montos negativos o iguales a cero en cualquier operación financiera: monto apostado, monto depositado y monto solicitado en un retiro deben ser estrictamente mayores a cero. De la misma manera, toda cuota de apuesta (cuota_actual) debe ser estrictamente mayor a 1.00, ya que una cuota igual o inferior a uno no representa una ganancia real para el apostador. Estas validaciones se aplican tanto en el método clean() de los formularios correspondientes (apuestas.forms.MontoApuestaForm, billetera.forms.DepositoForm, billetera.forms.SolicitudRetiroForm, apuestas.forms.MercadoOpcionForm) como mediante restricciones CHECK definidas directamente en las tablas de SQL Server, de manera que la integridad se garantiza incluso ante una inserción que no pase por la aplicación Django.

4.7 ATOMICIDAD DE LAS OPERACIONES FINANCIERAS

Toda operación que modifique el saldo de una billetera (creación de boleto, depósito, liquidación de evento con pago de boletos, aprobación de retiro) se ejecuta dentro de un bloque transaction.atomic() de Django, utilizando además select_for_update() sobre el registro de la billetera involucrada para evitar condiciones de carrera ante solicitudes concurrentes sobre la misma billetera. Esto garantiza que, ante cualquier error durante el proceso, ninguno de los cambios parciales se aplique a la base de datos, evitando así saldos inconsistentes.

4.8 SEGURIDAD DE SESIÓN Y CABECERAS HTTP

- Cookies de sesión configuradas como HttpOnly, con una duración máxima de cuatro horas y expiración al cerrar el navegador.

- Protección CSRF (Cross-Site Request Forgery) activa en todos los formularios del sistema mediante el middleware estándar de Django.

- Cabeceras de seguridad adicionales: X-Frame-Options en DENY (previene ataques de clickjacking), X-Content-Type-Options en nosniff, y filtro XSS del navegador habilitado.

-Variables SECURE_SSL_REDIRECT, SESSION_COOKIE_SECURE y CSRF_COOKIE_SECURE parametrizadas por entorno, para forzar HTTPS en producción.

4.9 RECUPERACIÓN DE CONTRASEÑA

El flujo de recuperación de contraseña exige, antes de permitir cualquier cambio, que el usuario demuestre conocer tanto el correo registrado como la respuesta correcta a la pregunta de seguridad que configuró al registrarse. Únicamente tras superar ambas verificaciones se genera un token (tbl_recuperacion_password.token, de tipo UUID) con una vigencia de treinta minutos y de un solo uso, el cual se envía exclusivamente al correo registrado en el sistema, nunca se muestra en pantalla. Por motivos de seguridad, el sistema no revela en ningún momento del paso inicial si el usuario o el correo ingresado existen en la base de datos, respondiendo siempre con un mensaje neutro.

LÓGICA DE NEGOCIO CRÍTICA

5.1 CREACIÓN DE BOLETOS (apuestas.views.crear_boleto)

Cuando el usuario confirma una apuesta desde el panel de usuario, la vista crear_boleto ejecuta la siguiente secuencia, en su totalidad dentro de una transacción atómica:

- Valida, mediante el formulario MontoApuestaForm, que el monto apostado sea mayor a cero y que la lista de opciones seleccionadas no esté vacía ni contenga identificadores repetidos.

- Recupera de la base de datos todas las opciones de apuesta seleccionadas y verifica que todas existan y que sus mercados y eventos correspondientes sigan abiertos y admitan apuestas.

- Calcula la cuota total del boleto como el producto de las cuotas individuales de cada selección, y la ganancia potencial como el monto apostado multiplicado por esa cuota total, redondeando ambos valores a dos decimales.

- Obtiene un bloqueo exclusivo (select_for_update) sobre el registro de la billetera del usuario, y verifica que el saldo disponible sea suficiente para cubrir el monto apostado.

- Descuenta el monto apostado del saldo disponible y lo suma al saldo retenido de la billetera.

- Registra el movimiento correspondiente en tbl_historial_saldo_billetera con tipo_operacion = DEBITO_APUESTA.

- Crea el registro en tbl_boleto con estado_boleto = PENDIENTE, y un registro en tbl_detalle_boleto por cada opción seleccionada, congelando en cuota_momento la cuota vigente en el instante de la apuesta (de manera que un cambio posterior de la cuota publicada no afecte boletos ya creados).

5.2 LIQUIDACIÓN DE EVENTOS Y PAGO AUTOMÁTICO DE BOLETOS (apuestas.views.admin_liquidar_evento)

Esta es la operación más compleja del sistema, también ejecutada en su totalidad dentro de una única transacción atómica:

- Registra (o actualiza) el resultado oficial del evento en tbl_resultado_evento, cambia el estado del evento a FINALIZADO y registra el cambio en tbl_historial_cambios_evento.

- Marca cada opción de apuesta del evento (tbl_opcion_apuesta.es_ganadora) según lo indicado por el operador en el formulario, y cambia el estado de todos los mercados del evento a LIQUIDADO.

- Identifica, mediante una consulta que atraviesa tbl_boleto → tbl_detalle_boleto → tbl_opcion_apuesta → tbl_mercado_evento, todos los boletos en estado PENDIENTE que incluyan al menos una selección relacionada con el evento que se está liquidando.

- Para cada uno de esos boletos, determina el conjunto completo de eventos involucrados en todas sus selecciones (relevante para boletos compuestos con selecciones de distintos eventos) y verifica que absolutamente todos esos eventos ya se encuentren en estado FINALIZADO; si algún evento del boleto todavía está pendiente, ese boleto se omite en esta ejecución y quedará pendiente de evaluación cuando se liquide su último evento faltante.

- Para los boletos cuyos eventos ya están todos finalizados, evalúa si la totalidad de sus selecciones (tbl_detalle_boleto) corresponden a opciones marcadas como es_ganadora = verdadero; si es así, el boleto pasa a estado GANADO, se obtiene un bloqueo exclusivo sobre la billetera correspondiente, se libera el monto apostado del saldo retenido y se acredita la ganancia_potencial al saldo disponible, registrando el movimiento en tbl_historial_saldo_billetera con tipo_operacion = CREDITO_PREMIO.

- Si al menos una selección del boleto no resultó ganadora, el boleto pasa a estado PERDIDO, y únicamente se libera (se descuenta) el monto apostado del saldo retenido, sin acreditar ninguna ganancia.

- En ambos casos se registra el cambio correspondiente en tbl_historial_estado_boleto, indicando el estado anterior, el nuevo estado y el motivo (liquidación automática de evento).

Debido a que todo este procedimiento ocurre dentro de una única transacción, si se produjera cualquier error durante la evaluación de alguno de los boletos afectados, ninguno de los cambios (incluyendo el resultado del evento ya registrado) se aplicaría a la base de datos, preservando la consistencia del sistema.

5.3 GESTIÓN DE RETIROS (billetera.views.admin_solicitudes_retiro)

Al aprobar una solicitud de retiro, el sistema vuelve a verificar, dentro de una transacción atómica y con bloqueo exclusivo sobre la billetera, que el usuario siga contando con saldo disponible suficiente (dado que pudo haber cambiado desde que se creó la solicitud, por ejemplo, por haberlo utilizado en una nueva apuesta). Si el saldo es suficiente, se descuenta del saldo disponible, se registra la transacción de tipo RETIRO y el movimiento correspondiente en el historial de saldo, y la solicitud pasa a estado PAGADA. Si el saldo ya no es suficiente, la aprobación se rechaza con un mensaje de error, sin modificar el estado de la solicitud. Al rechazar una solicitud, únicamente se actualiza su estado a RECHAZADA, sin ningún efecto sobre el saldo del usuario. Una solicitud que ya se encuentra en estado APROBADA, RECHAZADA o PAGADA no puede volver a procesarse.

ESTRUCTURA DE RUTAS (URLs) PRINCIPALES

Las rutas del sistema se organizan por aplicación, cada una con su propio archivo urls.py, incluido desde el archivo principal apuestas_deportivas/urls.py. A continuación se listan las rutas más relevantes:

6.1 PANEL PÚBLICO

GET / → vista inicio: cartelera pública de eventos de hoy y resultados de ayer.

6.2 CUENTAS

GET/POST /cuentas/registro/ → registro de usuario.

GET /cuentas/api/municipios/<id_departamento>/ → endpoint auxiliar (JSON) para la carga en cascada de municipios.

GET/POST /cuentas/login/ → inicio de sesión de usuario.

GET/POST /cuentas/login-admin/ → inicio de sesión administrativo.

GET /cuentas/logout/ → cierre de sesión.

GET/POST /cuentas/recuperar/ → solicitud de recuperación (usuario y correo).

GET/POST /cuentas/recuperar/pregunta/ → verificación de la pregunta de seguridad.

GET/POST /cuentas/recuperar/confirmar/<token>/ → definición de nueva contraseña.

6.3 APUESTAS

GET /panel-usuario/ → panel de usuario con cartelera de eventos abiertos.

POST /panel-usuario/crear-boleto/ → creación de un boleto de apuesta.

GET /panel-usuario/mis-boletos/ → historial de boletos del usuario.

GET /panel-administrativo/ → resumen del panel administrativo.

GET/POST /panel-administrativo/eventos/ → listado y alta de eventos.

GET/POST /panel-administrativo/eventos/<id_evento>/mercados/ → gestión de mercados y opciones de un evento.

POST /panel-administrativo/opciones/<id_opcion>/actualizar-cuota/ → actualización de una cuota publicada.

GET/POST /panel-administrativo/eventos/<id_evento>/liquidar/ → registro de resultado y liquidación del evento.

6.4 BILLETERA

GET/POST /panel-usuario/deposito/ → depósito de fondos.

GET/POST /panel-usuario/cuentas-bancarias/ → alta y listado de cuentas bancarias del usuario.

GET/POST /panel-usuario/retiro/ → solicitud de retiro e historial de solicitudes.

GET/POST /panel-administrativo/retiros/ → aprobación o rechazo de solicitudes de retiro.

6.5 ADMINISTRACIÓN TÉCNICA

/admin/ → panel de administración nativo de Django, con las 38 tablas registradas, disponible para el super administrador (control total sobre cualquier registro del sistema).

FORMULARIOS Y REGLAS DE VALIDACIÓN

Cada formulario del sistema se implementa como una clase de Django Forms, con validaciones propias en los métodos clean_<campo> y clean(). A continuación se resumen los formularios más relevantes:

- cuentas.forms.RegistroUsuarioForm: valida mayoría de edad a partir de la fecha de nacimiento, formato de teléfono, unicidad de correo y de nombre de usuario, formato permitido del nombre de usuario, coincidencia entre contraseña y su confirmación, y exige la selección de una pregunta de seguridad con su respuesta.

- cuentas.forms.LoginForm: formulario simple de usuario y contraseña, reutilizado tanto para el inicio de sesión de usuario como para el administrativo, diferenciándose únicamente por el conjunto de tipos de login contra los que se valida en la vista correspondiente.

-cuentas.forms.SolicitarRecuperacionForm, ResponderPreguntaForm y NuevaContrasenaForm: implementan, respectivamente, cada uno de los tres pasos del flujo de recuperación de contraseña descrito en la sección 4.9.

- apuestas.forms.MontoApuestaForm: valida que el monto apostado sea mayor a cero y no exceda un límite máximo razonable, y que la lista de opciones seleccionadas (recibida como una cadena de identificadores separados por coma) sea válida, no esté vacía y no contenga valores repetidos.

- apuestas.forms.EventoForm: valida que el equipo local y el equipo visitante de un nuevo evento no sean el mismo registro.

- apuestas.forms.MercadoOpcionForm: valida que la cuota ingresada sea mayor a 1.00.

- apuestas.forms.ResultadoEventoForm: valida que las puntuaciones registradas (local y visitante) sean valores enteros no negativos.

- billetera.forms.DepositoForm: valida que el monto a depositar sea mayor a cero y no exceda el límite máximo configurado por transacción.

- billetera.forms.SolicitudRetiroForm: valida que el monto solicitado sea mayor a cero.

CONFIGURACIÓN DEL PROYECTO (settings.py)

La configuración del proyecto se gestiona mediante variables de entorno, leídas a través de la librería python-decouple desde un archivo .env (no incluido en el control de versiones por motivos de seguridad; se entrega un archivo .env.example como plantilla de referencia). Los aspectos más relevantes configurados en settings.py son los siguientes:

- DATABASES: conexión a SQL Server mediante el motor django.db.backends.mssql, con codificación utf8mb4 y el modo SQL STRICT_TRANS_TABLES habilitado para evitar inserciones silenciosamente truncadas o inválidas.

- MIDDLEWARE: incluye, además de los middleware estándar de Django, los dos middleware propios descritos en las secciones 4.2 y 4.3 (auditoria.middleware.SeguridadPerimetralMiddleware y cuentas.middleware.ControlAccesoMiddleware).

-TEMPLATES: se agrega el context processor propio cuentas.context_processors.sesion_actual, el cual expone en todas las plantillas del sistema los datos de la sesión activa (si existe), evitando repetir esa lógica en cada vista.

- AUTH_PASSWORD_VALIDATORS: se mantienen los validadores estándar de Django para la robustez de contraseñas (similitud con los datos del usuario, longitud mínima, contraseñas comunes, contraseñas completamente numéricas).

- LANGUAGE_CODE y TIME_ZONE: configurados en es-gt y America/Guatemala respectivamente.

-SESSION_COOKIE_AGE, SESSION_COOKIE_HTTPONLY, SESSION_EXPIRE_AT_BROWSER_CLOSE: parámetros de duración y seguridad de la sesión, descritos en la sección 4.8.

- EMAIL_BACKEND y variables EMAIL_*: configuración del servidor de correo saliente utilizado para el envío de los enlaces de recuperación de contraseña; en entorno de desarrollo se utiliza el backend de consola (los correos se imprimen en la terminal), y en producción debe configurarse un backend SMTP real.

-SEGURIDAD_MAX_INTENTOS_LOGIN,
SEGURIDAD_MINUTOS_BLOQUEO_LOGIN,
SEGURIDAD_MAX_PETICIONES_POR_MINUTO,
SEGURIDAD_MINUTOS_VIGENCIA_TOKEN_RESET: parámetros propios de seguridad del sistema, ajustables sin modificar el código fuente de las vistas.

INSTALACIÓN Y DESPLIEGUE

9.1 REQUISITOS PREVIOS

Python 3.12 – 3.14.

SQL Server 2022 o superior.

pip y soporte para entornos virtuales (venv).

Una cuenta de correo (o servidor SMTP) habilitada para el envío de los correos de recuperación de contraseña en producción.

9.2 CREACIÓN DEL ENTORNO VIRTUAL E INSTALACIÓN DE DEPENDENCIAS

cd apuestas_deportivas

python3 -m venv venv

source venv/bin/activate (en Windows: venv\Scripts\activate)

pip install -r requirements.txt

El archivo requirements.txt incluye: Django 4.2.30, mssql-python 2.2.4, python-decouple 3.8 y Pillow 12.1.1.

9.3 CONFIGURACIÓN DE VARIABLES DE ENTORNO

cp .env.example .env

Edite el archivo .env recién creado para indicar, como mínimo, los datos de conexión a SQL Server (DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT) y, si se desea probar el envío real de correos, los datos del servidor SMTP (EMAIL_HOST, EMAIL_PORT, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD). En producción, además, debe generarse un valor único y aleatorio para DJANGO_SECRET_KEY, establecer DJANGO_DEBUG=False, definir correctamente DJANGO_ALLOWED_HOSTS con el o los dominios del sistema, y activar DJANGO_SSL_REDIRECT=True si el sitio se sirve bajo HTTPS.

9.4 CREACIÓN DE LA BASE DE DATOS

Existen dos formas equivalentes de crear la base de datos y sus 38 tablas:

Opción A — ejecutar directamente el script SQL entregado (recomendada para producción, ya que el DBA puede revisar exactamente el DDL que se ejecutará):

mssql-python -u root -p < sql/bd_apuestas_deportivas.sql

python manage.py migrate --fake

Opción B — dejar que Django cree las tablas a partir de los modelos (recomendada para entornos de desarrollo):

mssql-python -u root -p -e "CREATE DATABASE bd_apuestas_deportivas CHARACTER SET utf8mb4;"

python manage.py migrate

9.5 CARGA DE DATOS INICIALES

El proyecto incluye el comando de gestión cargar_datos_iniciales (definido en catalogos/management/commands/cargar_datos_iniciales.py), el cual crea de forma idempotente (puede ejecutarse más de una vez sin duplicar datos) los siguientes registros base:

- Estados genéricos del catálogo tbl_estado (ACTIVO, INACTIVO, BLOQUEADO, ACTIVA, SUSPENDIDA).

- País Guatemala y sus departamentos, cada uno con un municipio de cabecera de referencia.

- Preguntas de seguridad predefinidas para la recuperación de contraseña.

- Tipos de mercado y métodos de pago más comunes.

- Un deporte (Fútbol, Baloncesto) y una liga de referencia.

- Un super administrador inicial, con una contraseña generada que se muestra una única vez en la terminal al ejecutar el comando, y que debe cambiarse de inmediato tras el primer acceso.

python manage.py cargar_datos_iniciales

9.6 EJECUCIÓN DEL SERVIDOR

Para un entorno de desarrollo o pruebas:

python manage.py runserver

Para producción, se recomienda servir la aplicación mediante un servidor WSGI robusto (por ejemplo, Gunicorn) detrás de un servidor web como Nginx, el cual además debe encargarse de servir los archivos estáticos recolectados mediante el comando python manage.py collectstatic, así como de la terminación TLS/HTTPS.

9.7 CONSIDERACIONES ADICIONALES PARA PRODUCCIÓN

- Establecer DJANGO_DEBUG=False y configurar correctamente DJANGO_ALLOWED_HOSTS; dejar DEBUG activo en producción expone información sensible de la aplicación ante cualquier error.

- Servir el sitio exclusivamente bajo HTTPS, activando DJANGO_SSL_REDIRECT, SESSION_COOKIE_SECURE y CSRF_COOKIE_SECURE.

- Configurar un servidor SMTP real (no el backend de consola) para que la recuperación de contraseña funcione correctamente.

- Definir una política de respaldo (backup) periódico de la base de datos SQL Server, dado que contiene información financiera de los usuarios.

- Revisar y ajustar, según el volumen de tráfico esperado, los parámetros SEGURIDAD_MAX_PETICIONES_POR_MINUTO y SEGURIDAD_MAX_INTENTOS_LOGIN.

- Cambiar de inmediato la contraseña del super administrador inicial generada por el comando cargar_datos_iniciales.

PRUEBAS REALIZADAS DURANTE EL DESARROLLO

Durante el desarrollo del sistema se realizaron, mediante el cliente de pruebas de Django (Client de django.test), verificaciones funcionales sobre los siguientes escenarios, confirmando en todos los casos el comportamiento esperado:

- Carga correcta (código HTTP 200) de las rutas principales del panel público, de inicio de sesión, de registro y de recuperación de contraseña.

- Flujo completo de inicio de sesión de usuario y de administrador, con redirección correcta al panel correspondiente.

- Flujo completo de depósito de fondos, verificando el incremento correcto del saldo disponible de la billetera.

- Flujo completo de creación de un boleto de apuesta, verificando el traslado correcto del monto apostado desde el saldo disponible hacia el saldo retenido, y el cálculo correcto de la cuota total y la ganancia potencial.

- Flujo completo de liquidación de un evento marcando una opción como ganadora, verificando que el boleto correspondiente cambie automáticamente a estado GANADO y que la ganancia potencial se acredite correctamente al saldo disponible del usuario (se verificó, con datos reales, el caso de un saldo inicial de Q600.00, una apuesta de Q50.00 a cuota 1.85, resultando en un saldo final de Q642.50 tras la liquidación, coincidiendo exactamente con el cálculo esperado).

- Bloqueo efectivo, por parte del middleware de seguridad perimetral, de un intento de inyección SQL enviado en el campo de nombre de usuario del formulario de inicio de sesión, confirmando además el registro correspondiente en la bitácora de seguridad.

- Rechazo efectivo de montos negativos tanto en el formulario de depósito como en el de creación de boleto, con el mensaje de validación correspondiente mostrado al usuario.

- Activación efectiva del límite de solicitudes por minuto (rate limiting), devolviendo el código de respuesta HTTP 429 ante una ráfaga de solicitudes desde la misma dirección IP, y registrando el evento correspondiente en la bitácora de seguridad.

Se recomienda complementar estas verificaciones funcionales con pruebas automatizadas formales (por ejemplo, mediante la clase TestCase de Django) organizadas por aplicación, así como con pruebas de carga antes de un despliegue a producción con tráfico real.

MANTENIMIENTO Y EXTENSIÓN FUTURA

El sistema fue diseñado de manera modular, por lo que su extensión futura puede realizarse de forma localizada, sin afectar al resto del código. Algunas recomendaciones para su mantenimiento y evolución:

- Nuevos tipos de mercado de apuesta: basta con agregar un nuevo registro en tbl_tipo_mercado desde el panel administrativo o el panel técnico; no se requiere ningún cambio de código, ya que los mercados y sus opciones se configuran de forma totalmente dinámica.

- Nuevos roles administrativos: actualmente el campo rol_administrativo de tbl_administrador admite los valores OPERADOR_EVENTOS, CAJERO y AUDITOR; agregar un nuevo rol requiere extender esa enumeración tanto en el modelo (apps/cuentas/models.py) como en la correspondiente migración de base de datos, y ajustar las verificaciones de permisos en las vistas administrativas afectadas.

- Notificaciones automáticas por correo: el sistema ya cuenta con la infraestructura de envío de correo utilizada para la recuperación de contraseña (función send_mail de Django); esta misma infraestructura puede reutilizarse para notificar, por ejemplo, cuando un boleto resulta ganador o cuando una solicitud de retiro es aprobada.

- Autenticación de doble factor: el modelo tbl_super_administrador ya reserva el campo llave_mfa_secret para una futura implementación de autenticación de doble factor (TOTP) sobre las cuentas de mayor privilegio.

Antes de modificar la lógica de liquidación de boletos (sección 5.2) o de creación de boletos (sección 5.1), se recomienda encarecidamente comprender a fondo el uso de transacciones atómicas y bloqueos (select_for_update) ya implementado, dado que cualquier cambio que omita esa protección podría introducir inconsistencias en el saldo de los usuarios bajo uso concurrente.